# CSL351 : COMPUTER NETWORKS
# Assignment - 5 (Network Simulator3)

**Name - Ashish Ranjan**
**ID no. - 12340370**

**Drive Link for the Pcap files and the codes :**
https://drive.google.com/drive/folders/1A6FQmUbOtCt_uZ_9EQX_AFSc3z818H65?usp=drive_link

**Instruction for the Drive link :**
All the codes and files are in separate folder all the que1.cc to que7.cc codes are in ‘codes’ folder, the shell script is in ‘Shell Script’ folder, xml file in ‘XML’ folder, the pcap for ques3 in ’12340370_Assignment_5’ folder and the pcap for ques 7 is in ‘Pcap_que7’ folder.

**Instruction for the zip file :**
For every question we have separate folder with name as Q<ques_number> which contains the .cc file and other .xml files and the .pcap files if required.

**Report from the next page :**

**Que 1.**

**Explanation :**

We created two fundamental nodes, with a point to point network,then we installed the internet stack for the TCP/UDP and then we assigned the IP address for the nodes.

**Functions for the Topology :**

1. Node Setup : nodes.Create(2)
2. Configuration for Link : created the p2p object under the PointoPointHelper class
   > Bandwidth : set using p2p.SetDeviceAttribute("DataRate", ...)
   > Latency : set using p2p.SetChannelAttribute("Delay", ...)

**Functions for Network Stack and IP Address :**

1. Protocol Installation : using the InternetStackHelper class to create the stack object using stack.Install(nodes).
2. IP Address Assignment : using the address object from the Ipv4AddressHelper class
   > Base Network : address.SetBase("10.1.1.0", "255.255.255.0").
   > Assignment : The IP addresses from this network are then assigned to the link devices using the address.Assign(devices) function.

Printing the logs for understanding where the code execution is currently at.

**COMMAND USED :** ./ns3 build
./ns3 run scratch/que1

**CODE :**

```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"
#include "ns3/netanim-module.h"

using namespace ns3;
using namespace std;
NS_LOG_COMPONENT_DEFINE ("Assignment5");
```

```
int main(int argc,char *argv[]){
bool trace=true;
bool anim=false;

LogComponentEnable ("Assignment5", LOG_LEVEL_INFO);
string bandwidth="1Mbps";
string latency="1ms";
CommandLine cmd;
cmd.AddValue("bandwidth","link data rate",bandwidth);
cmd.AddValue("latency","latency of the Link",latency);
cmd.Parse(argc, argv);
NS_LOG_INFO ("Creating nodes.");
NodeContainer nodes;
nodes.Create (2);
NS_LOG_INFO ("Creating channels.");
PointToPointHelper p2p;
p2p.SetDeviceAttribute ("DataRate", StringValue(bandwidth));
p2p.SetChannelAttribute ("Delay", StringValue(latency));

NetDeviceContainer devices;
devices = p2p.Install (nodes);

InternetStackHelper stack;
stack.Install (nodes);

NS_LOG_INFO ("assigning IP addr");
Ipv4AddressHelper address;
address.SetBase ("10.1.1.0", "255.255.255.0");
Ipv4InterfaceContainer interfaces = address.Assign (devices);
if(trace){
 p2p.EnablePcapAll("p2p");
}
if(anim){
 AnimationInterface anim("que1.xml");
 anim.SetConstantPosition(nodes.Get(0), 10.0, 20.0);
 anim.SetConstantPosition(nodes.Get(1), 60.0, 20.0);
}
Simulator::Stop(Seconds(60.0));
Simulator::Run();
Simulator::Destroy();
```

```
return 0;
}
```

## Output Terminal :

```
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV:~/ns-allinone-3.44/ns-3.44$ ./ns3 run scratch/que1
Creating nodes.
Creating channels.
assigning IP addr
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV:~/ns-allinone-3.44/ns-3.44$
```

## Result :

The successful creation of the PCAP files validates that the Point-to-Point link was properly set up with the default values for the bandwidth and latency.

**Que 2.**

**Explanation :**

Now we create the application layer traffic using the UDP,  and a UDP server is created on Node1 which receives packets , and our UDP client which is on Node0 sends data to the server at a constant rate.
We accepted the arguments regarding the maximum packets, interval, packet size via command line using the cmd.addValue().

**UDP Server Configuration :**

1. We used the UdpServerHelper class for the server object on port 8790.
2. The server is installed on Node 1 (using nodes.Get(1)) under the appserver object.
3. For the server start we used appserver.Start(), and to stop the server we have used appserver.Stop()

**UDP Client Configuration :**

1. We used client.Install(nodes.Get(0)) to instantiate the client with out destination IP.
2. The client is installed on Node0 using client.Install(nodes.Get(0)).
3. Attributes Data :
   > Total packets  was set using client.SetAttribute("MaxPackets", ...).
   > Time interval between packets was set using client.SetAttribute("Interval", ...).
   > Size per  packet was set using client.SetAttribute("PacketSize", ...).

For running the simulation and logging the parameters and method remains same as that of the previous question.

**COMMAND USED :**

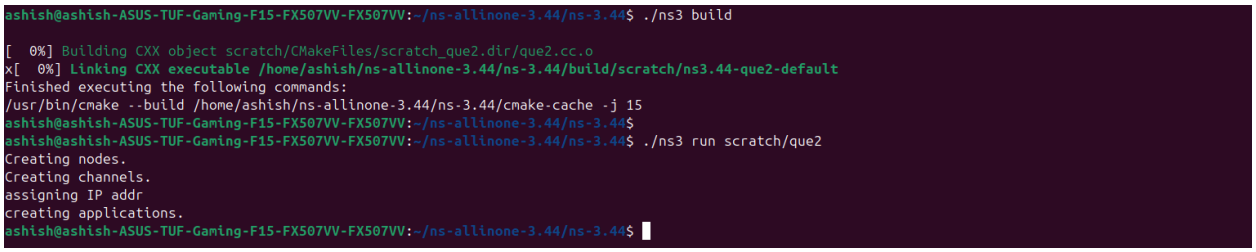
./ns3 build
./ns3 run scratch/que2

**CODE :**

```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"
#include "ns3/netanim-module.h"


using namespace ns3;
using namespace std;
NS_LOG_COMPONENT_DEFINE ("Assignment5");
int main(int argc,char *argv[]){
bool trace=true;
bool anim=false;

LogComponentEnable ("Assignment5", LOG_LEVEL_INFO);
string bandwidth="1Mbps";
string latency="1ms";
uint32_t packetsize=1024;
uint32_t maxipackets=100;
string interval="1s";
CommandLine cmd;
cmd.AddValue("bandwidth","link data rate",bandwidth);
cmd.AddValue("latency","latency of the Link",latency);
cmd.AddValue ("packetsize", "Size of packets", packetsize);
cmd.AddValue ("maxpackets", "Maximum number of packets", maxipackets);
cmd.AddValue ("interval", "Interval between packets", interval);

cmd.Parse(argc, argv);
NS_LOG_INFO ("Creating nodes.");
NodeContainer nodes;
nodes.Create (2);
NS_LOG_INFO ("Creating channels.");
PointToPointHelper p2p;
p2p.SetDeviceAttribute ("DataRate", StringValue(bandwidth));
p2p.SetChannelAttribute ("Delay", StringValue(latency));

NetDeviceContainer devices;
devices = p2p.Install (nodes);
```

```
InternetStackHelper stack;
stack.Install (nodes);

NS_LOG_INFO ("assigning IP addr");
Ipv4AddressHelper address;
address.SetBase ("10.1.1.0", "255.255.255.0");
Ipv4InterfaceContainer interfaces = address.Assign (devices);

NS_LOG_INFO ("creating applications.");
uint16_t port=8790;
UdpServerHelper server(port);
ApplicationContainer appserver=server.Install(nodes.Get(1));
appserver.Start(Seconds(1.0));
appserver.Stop(Seconds(60.0));
UdpClientHelper client (interfaces.GetAddress (1), port);
client.SetAttribute("MaxPackets", UintegerValue (maxipackets));
client.SetAttribute("Interval", TimeValue (Time(interval)));
client.SetAttribute("PacketSize", UintegerValue (packetsize));
ApplicationContainer clientApp = client.Install (nodes.Get(0));
clientApp.Start (Seconds (2.0));
clientApp.Stop (Seconds (60.0));

if(trace){
 p2p.EnablePcapAll("p2p");
}
if(anim){
 AnimationInterface anim("que1.xml");
 anim.SetConstantPosition(nodes.Get(0), 10.0, 20.0);
 anim.SetConstantPosition(nodes.Get(1), 60.0, 20.0);
}
Simulator::Stop(Seconds(60.0));
Simulator::Run();
Simulator::Destroy();
return 0;
}
```

## Output Terminal :




## Result :

the client application successfully transmitted data, which the server received without any reported errors.

It confirms the correct and operational status of the fundamental connection between the sender and the receiver.

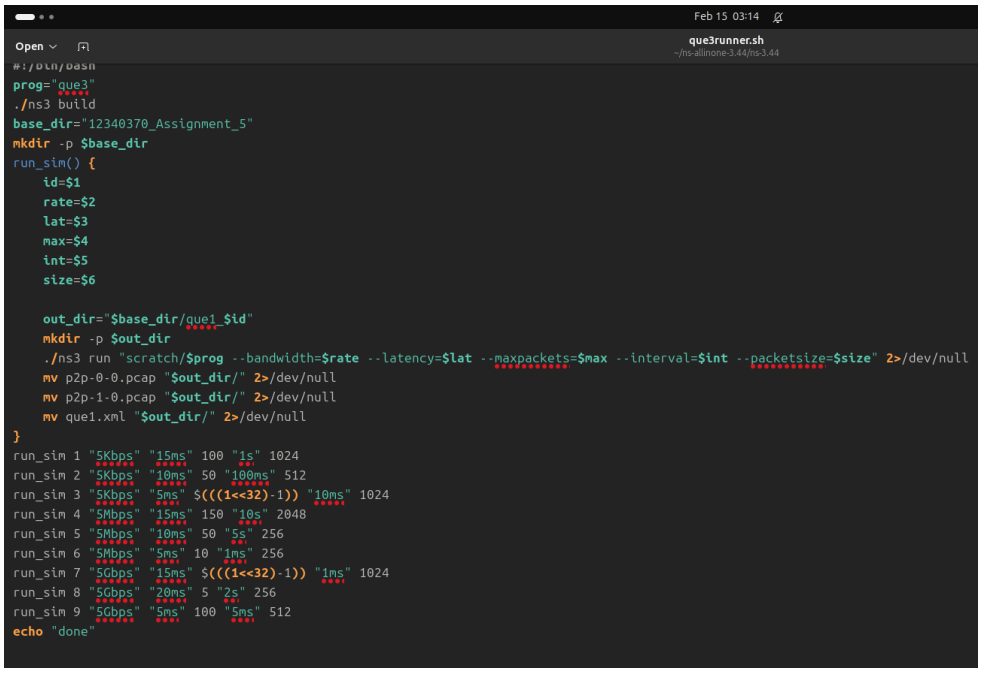
**Que 3.**

**Explanation :**

We have used the same code as that of ques2. For running the same file multiple times to generate the pcap file for analysis we have used the shell script.

**Shell Script Explanation :**

I used a shell script to automate the process. It runs the simulation nine times with different parameters bandwidth, latency, packet size and organizes the output pcap files into folders. For each simulation, it organizes the resulting pcap files into dedicated subdirectories within a main “12340370_assignment_5” folder.




**Run Configuration :**

We ran the simulator for 60 seconds for each execution using the Simulator::Stop(Seconds(60.0)).

Then we opened the wireshark to get the end-to-end throughput as well as total packets transmitted.

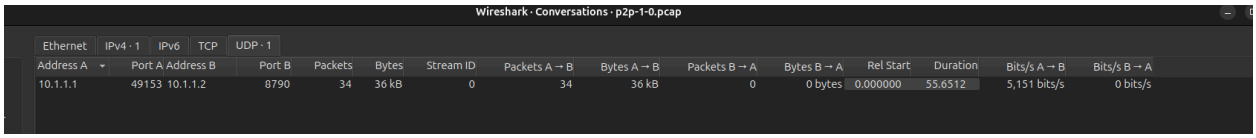
**COMMAND USED :**

chmod +x que3runner.sh

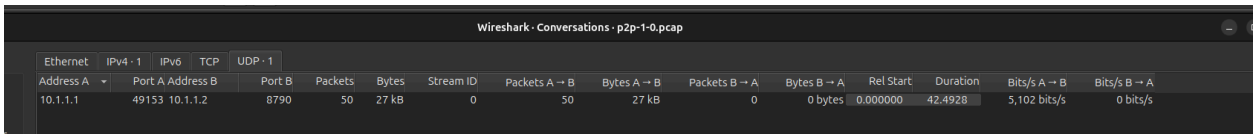
./que3runner.sh

## Output from the Wireshark :

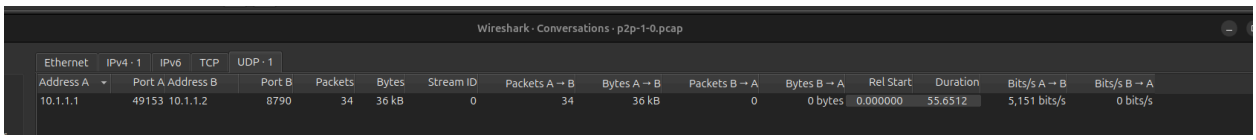
Row 1 :



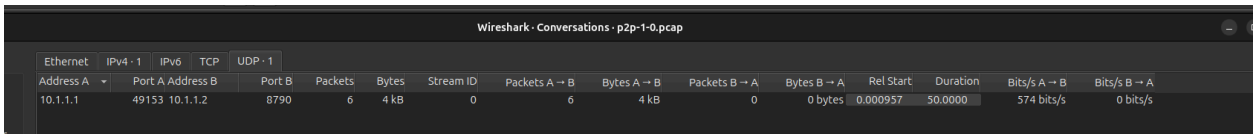

Row 2 :



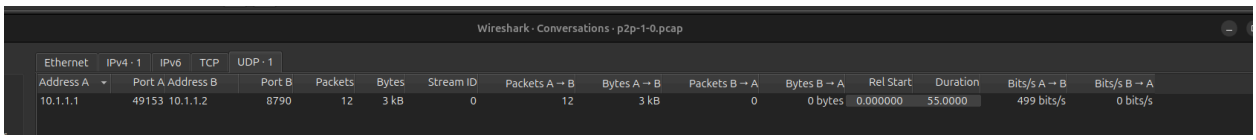

Row 3 :




Row 4 :

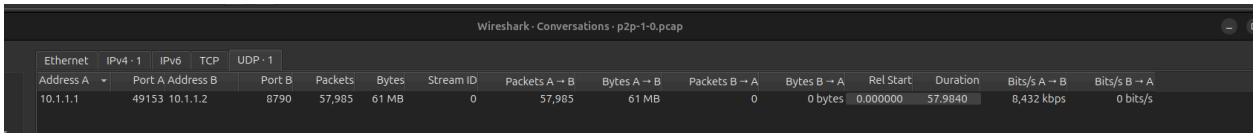



Row 5 :

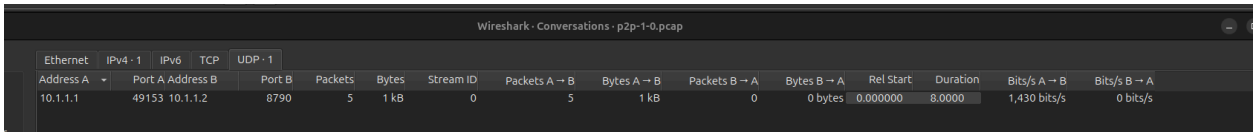



Row 6 :


Row 7 :




Row 8 :




Row 9 :

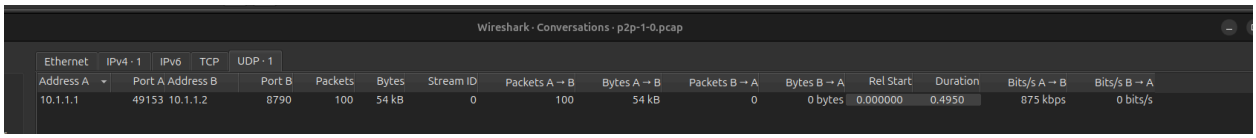



**Table Data :**

| Row | Bandwidth | Latency | Max Packets | Interval | Packet Size | Throughput | Packets |
|---|---|---|---|---|---|---|---|
| 1 | 5 Kbps | 15 ms | 100 | 1 s | 1024 | 5.151 Kbps | 34 |
| 2 | 5 Kbps | 10 ms | 50 | 100 ms | 512 | 5.102 Kbps | 50 |
| 3 | 5 Kbps | 5 ms | Unlimited | 10 ms | 1024 | 5.151 Kbps | 34 |
| 4 | 5 Mbps | 15 ms | 150 | 10 s | 2048 | 0.574 Kbps | 6 |
| 5 | 5 Mbps | 10 ms | 50 | 5 s | 256 | 0.499 Kbps | 12 |
| 6 | 5 Mbps | 5 ms | 10 | 1 ms | 256 | 2.542 Mbps | 10 |
| 7 | 5 Gbps | 15 ms | Unlimited | 1 ms | 1024 | 8.432 Mbps | 57,985 |
| 8 | 5 Gbps | 20 ms | 5 | 2 s | 256 | 1.430 Kbps | 5 |
| 9 | 5 Gbps | 5 ms | 100 | 5 ms | 512 | 875 kbps | 100 |

**Shell Script File Link :**
https://drive.google.com/drive/folders/1_A7Y7D4rZ4ZUze7AWRyFdNhcU2ONEREV?usp=drive_link

**Generated Folder with All the pcap files :**
https://drive.google.com/drive/folders/1Ki9I53_iawsxCAu_68fgI3kSKuKhz2tw?usp=drive_link

**Results :**

1. Higher latency delayed the data arrival but did not change the link's speed.
2. On slow links 5 Kbps, a very small interval (fast sending) causes the throughput to hit a ceiling.
3. Higher bandwidth increases speed, while higher latency slows it down.
4. The best performance came from the combination of high bandwidth, low delay, and large packet sizes.

**Que 4.**

**Explanation :**

Two concurrent application flows the original UdpClient Server and a new UdpEchoClient Server pair were added to the two nodes to show multiple applications sharing the network link.

Configuration of Network :

1. The established UdpClient/Server flow continued to operate on the network link.
2. A second concurrent flow was introduced with a UdpEchoServer installed on Node 1 and a UdpEchoClient installed on Node 0.
3. To avoid port conflict, the Echo Server used a separate port (8793).
4. The new Echo Client was configured to use the same traffic parameters (MaxPackets, Interval, PacketSize) as the initial UDP client like that of previous questions.
5. Both client applications were scheduled to start at the 2.0 second mark and run until the simulation was terminated at 60.0 seconds.

Logging is same as previous questions.

**COMMAND USED :**

./ns3 build

./ns3 run scratch/que4

**CODE :**

```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"
#include "ns3/netanim-module.h"


using namespace ns3;
```

```
using namespace std;
NS_LOG_COMPONENT_DEFINE ("Assignment5");
int main(int argc,char *argv[]){
bool trace=true;
bool anim=false;

LogComponentEnable ("Assignment5", LOG_LEVEL_INFO);
string bandwidth="1Mbps";
string latency="1ms";
uint32_t packetsize=1024;
uint32_t maxipackets=10;
string interval="1s";
CommandLine cmd;
cmd.AddValue("bandwidth","link data rate",bandwidth);
cmd.AddValue("latency","latency of the Link",latency);
cmd.AddValue ("packetsize", "Size of packets", packetsize);
cmd.AddValue ("maxpackets", "Maximum number of packets", maxipackets);
cmd.AddValue ("interval", "Interval between packets", interval);

cmd.Parse(argc, argv);

Time::SetResolution (Time::NS);
NS_LOG_INFO ("Creating nodes.");
NodeContainer nodes;
nodes.Create (2);
NS_LOG_INFO ("Creating channels.");
PointToPointHelper p2p;
p2p.SetDeviceAttribute ("DataRate", StringValue(bandwidth));
p2p.SetChannelAttribute ("Delay", StringValue(latency));

NetDeviceContainer devices;
devices = p2p.Install (nodes);

InternetStackHelper stack;
stack.Install (nodes);

NS_LOG_INFO ("assigning IP addr");
Ipv4AddressHelper address;
address.SetBase ("10.1.1.0", "255.255.255.0");
Ipv4InterfaceContainer interfaces = address.Assign (devices);
```

```
NS_LOG_INFO ("creating applications.");
uint16_t port=8790;
UdpServerHelper server(port);
ApplicationContainer appserver=server.Install(nodes.Get(1));
appserver.Start(Seconds(1.0));
appserver.Stop(Seconds(60.0));
UdpClientHelper client (interfaces.GetAddress (1), port);
client.SetAttribute("MaxPackets", UintegerValue (maxipackets));
client.SetAttribute("Interval", TimeValue (Time(interval)));
client.SetAttribute("PacketSize", UintegerValue (packetsize));
ApplicationContainer clientApp = client.Install (nodes.Get(0));
clientApp.Start (Seconds (2.0));
clientApp.Stop (Seconds (60.0));


NS_LOG_INFO ("creating applications with udp echo helper");
uint16_t newport = 8793;

 UdpEchoServerHelper echoServer(newport);
 ApplicationContainer echoServerApps = echoServer.Install(nodes.Get(1));
 echoServerApps.Start(Seconds(1.0));
 echoServerApps.Stop(Seconds(60.0));


 UdpEchoClientHelper echoClient(interfaces.GetAddress(1), newport);
 echoClient.SetAttribute("MaxPackets", UintegerValue(maxipackets));
 echoClient.SetAttribute("Interval", TimeValue(Time(interval)));
 echoClient.SetAttribute("PacketSize", UintegerValue(packetsize));
 ApplicationContainer echoapps = echoClient.Install(nodes.Get(0));
 echoapps.Start(Seconds(2.0));
 echoapps.Stop(Seconds(60.0));
if(trace){
 p2p.EnablePcapAll("p2p");
}
if(anim){
 AnimationInterface anim("que1.xml");
 anim.SetConstantPosition(nodes.Get(0), 10.0, 20.0);
 anim.SetConstantPosition(nodes.Get(1), 60.0, 20.0);
}
```

```
Simulator::Stop(Seconds(60.0));
Simulator::Run();
Simulator::Destroy();
return 0;
}
```

**Output Terminal :**

```
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV:~/ns-allinone-3.44/ns-3.44$ ./ns3 run scratch/que4
Creating nodes.
Creating channels.
assigning IP addr
creating applications.
creating applications with udp echo helper
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV:~/ns-allinone-3.44/ns-3.44$
```

**Results :**

1. Two distinct applications (Standard UDP and UdpEcho) were executed on the same pair of nodes.
2. The use of separate port numbers (8790 and 8793) ensured the two concurrent application flows did not cause any issues with each other.

**Que 5.**

**Explanation :**

We used the same code as that of ques 4. The goal was to verify that the transport layer correctly multiplexes traffic to the specific server instances under load.

**Simulation Parameters:**

1. Bandwidth: 5Mbps
2. Latency: 2ms
3. Interval: 10ms
4. Packet size: Unlimited.
5. Duration: the simulation ran for 30 seconds.

**COMMAND USED :**

./ns3 build

./ns3 run “scratch/que5 --bandwidth=5Mbps --latency=2ms --maxpackets=4294967295 --interval=2ms --packetsize=1024”

**CODE :**

```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"
#include "ns3/netanim-module.h"



using namespace ns3;
using namespace std;
NS_LOG_COMPONENT_DEFINE ("Assignment5");
int main(int argc,char *argv[]){
bool trace=true;
bool anim=false;

LogComponentEnable ("Assignment5", LOG_LEVEL_INFO);
string bandwidth="1Mbps";
string latency="1ms";
uint32_t packetsize=1024;
uint32_t maxipackets=10;
```

```
string interval="1s";
CommandLine cmd;
cmd.AddValue("bandwidth","link data rate",bandwidth);
cmd.AddValue("latency","latency of the Link",latency);
cmd.AddValue ("packetsize", "Size of packets", packetsize);
cmd.AddValue ("maxpackets", "Maximum number of packets", maxipackets);
cmd.AddValue ("interval", "Interval between packets", interval);

cmd.Parse(argc, argv);

Time::SetResolution (Time::NS);
NS_LOG_INFO ("Creating nodes.");
NodeContainer nodes;
nodes.Create (2);
NS_LOG_INFO ("Creating channels.");
PointToPointHelper p2p;
p2p.SetDeviceAttribute ("DataRate", StringValue(bandwidth));
p2p.SetChannelAttribute ("Delay", StringValue(latency));

NetDeviceContainer devices;
devices = p2p.Install (nodes);

InternetStackHelper stack;
stack.Install (nodes);

NS_LOG_INFO ("assigning IP addr");
Ipv4AddressHelper address;
address.SetBase ("10.1.1.0", "255.255.255.0");
Ipv4InterfaceContainer interfaces = address.Assign (devices);

NS_LOG_INFO ("creating applications.");
uint16_t port=8790;
UdpServerHelper server(port);
ApplicationContainer appserver=server.Install(nodes.Get(1));
appserver.Start(Seconds(1.0));
appserver.Stop(Seconds(30.0));
UdpClientHelper client (interfaces.GetAddress (1), port);
client.SetAttribute("MaxPackets", UintegerValue (maxipackets));
client.SetAttribute("Interval", TimeValue (Time(interval)));
client.SetAttribute("PacketSize", UintegerValue (packetsize));
```

```
ApplicationContainer clientApp = client.Install (nodes.Get(0));
clientApp.Start(Seconds (2.0));
clientApp.Stop(Seconds (30.0));


NS_LOG_INFO ("creating applications with udp echo helper");
uint16_t newport = 8793;

 UdpEchoServerHelper echoServer(newport);
 ApplicationContainer echoServerApps = echoServer.Install(nodes.Get(1));
 echoServerApps.Start(Seconds(1.0));
 echoServerApps.Stop(Seconds(30.0));


 UdpEchoClientHelper echoClient(interfaces.GetAddress(1), newport);
  echoClient.SetAttribute("MaxPackets", UintegerValue(maxipackets));
 echoClient.SetAttribute("Interval", TimeValue(Time(interval)));
 echoClient.SetAttribute("PacketSize", UintegerValue(packetsize));
 ApplicationContainer echoapps = echoClient.Install(nodes.Get(0));
 echoapps.Start(Seconds(2.0));
 echoapps.Stop(Seconds(30.0));
if(trace){
 p2p.EnablePcapAll("p2p");
}
if(anim){
 AnimationInterface anim("que1.xml");
 anim.SetConstantPosition(nodes.Get(0), 10.0, 20.0);
 anim.SetConstantPosition(nodes.Get(1), 60.0, 20.0);
}
Simulator::Stop(Seconds(30.0));
Simulator::Run();
Simulator::Destroy();
return 0;
}
```

## Output Terminal :

```
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV:~/ns-allinone-3.44/ns-3.44$ ./ns3 run "scratch/que5 --bandwidth=5Mbps --latency=2ms --maxpackets=4294967295 --interval=10ms --packetsize=1024"
Creating nodes.
Creating channels.
assigning IP addr
creating applications.
creating applications with udp echo helper
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV:~/ns-allinone-3.44/ns-3.44$
```

**Results :**

1. The network consistently managed the heavy load, even with continuous and rapid transmission of an unlimited number of packets.
2. Wireshark analysis confirmed the correct transport layer multiplexing, showing the traffic for the Echo server was distinctly separated from the Standard server.

**Que 6.**

**Explanation :**

In this question,the simulation was enhanced to support graphical visualization via the netnaim tool. It required configuring a static mobility model to define the precsie coordinates for each node and generating a specific xml file.

**Functions used:**

1. ns3::ConstantPositionMobilityModel via MobilityHelper to define fixed node coordinates, which were explicitly assigned using anim.SetConstantPosition for a readable diagram.

**netanim:**

1. The file was created with the name "que6.xml". Which contains the simulation events node co-od for the topology diagram we have created.

**Coordinates used:**

1. Node 0 co-od (10.0, 20.0).
2. Node 1 co-od (60.0, 20.0).

**COMMAND USED :**

**Inside ns3 folder :**

./ns3 build

./ns3 run scratch/que6

**Inside netanim folder**

cmake .

Make

cd bin

./netanim

**CODE :**

```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"
```

```
#include "ns3/netanim-module.h"
#include "ns3/mobility-module.h"

using namespace ns3;
using namespace std;
NS_LOG_COMPONENT_DEFINE ("Assignment5");
int main(int argc,char *argv[]){
bool trace=true;

LogComponentEnable ("Assignment5", LOG_LEVEL_INFO);
string bandwidth="1Mbps";
string latency="1ms";
uint32_t packetsize=1024;
uint32_t maxipackets=10;
string interval="1s";
CommandLine cmd;
cmd.AddValue("bandwidth","link data rate",bandwidth);
cmd.AddValue("latency","latency of the Link",latency);
cmd.AddValue ("packetsize", "Size of packets", packetsize);
cmd.AddValue ("maxpackets", "Maximum number of packets", maxipackets);
cmd.AddValue ("interval", "Interval between packets", interval);

cmd.Parse(argc, argv);

Time::SetResolution (Time::NS);
NS_LOG_INFO ("Creating nodes.");
NodeContainer nodes;
nodes.Create (2);

NS_LOG_INFO ("Creating mobility forr nodes.");
MobilityHelper mobility;
mobility.SetMobilityModel ("ns3::ConstantPositionMobilityModel");
mobility.Install(nodes);


NS_LOG_INFO ("Creating channels.");
PointToPointHelper p2p;
p2p.SetDeviceAttribute ("DataRate", StringValue(bandwidth));
p2p.SetChannelAttribute ("Delay", StringValue(latency));
```

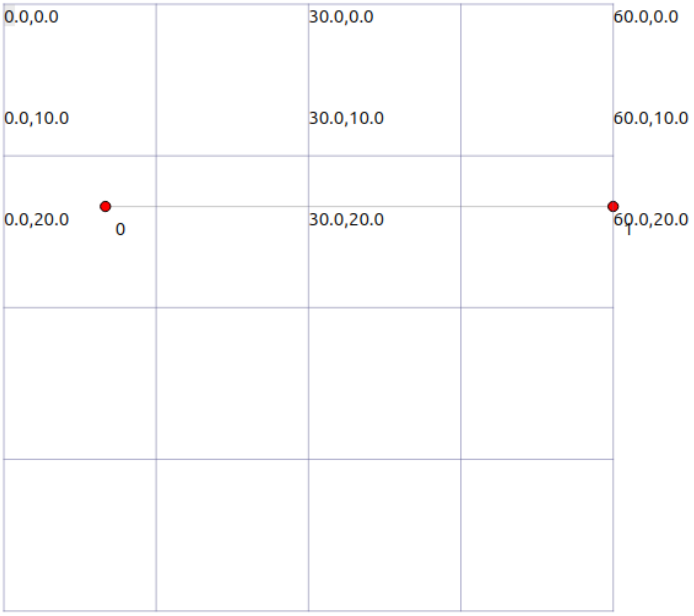
```
NetDeviceContainer devices;
devices = p2p.Install (nodes);

InternetStackHelper stack;
stack.Install (nodes);

NS_LOG_INFO ("assigning IP addr");
Ipv4AddressHelper address;
address.SetBase ("10.1.1.0", "255.255.255.0");
Ipv4InterfaceContainer interfaces = address.Assign (devices);

NS_LOG_INFO ("creating applications.");
uint16_t port=8790;
UdpServerHelper server(port);
ApplicationContainer appserver=server.Install(nodes.Get(1));
appserver.Start(Seconds(1.0));
appserver.Stop(Seconds(31.0));
UdpClientHelper client (interfaces.GetAddress (1), port);
client.SetAttribute("MaxPackets", UintegerValue (maxipackets));
client.SetAttribute("Interval", TimeValue (Time(interval)));
client.SetAttribute("PacketSize", UintegerValue (packetsize));
ApplicationContainer clientApp = client.Install (nodes.Get(0));
clientApp.Start(Seconds(1.0));
clientApp.Stop(Seconds(31.0));


NS_LOG_INFO ("creating applications with udp echo helper");
uint16_t newport = 8793;

 UdpEchoServerHelper echoServer(newport);
 ApplicationContainer echoServerApps = echoServer.Install(nodes.Get(1));
 echoServerApps.Start(Seconds(1.0));
 echoServerApps.Stop(Seconds(31.0));


 UdpEchoClientHelper echoClient(interfaces.GetAddress(1), newport);
  echoClient.SetAttribute("MaxPackets", UintegerValue(maxipackets));
 echoClient.SetAttribute("Interval", TimeValue(Time(interval)));
 echoClient.SetAttribute("PacketSize", UintegerValue(packetsize));
 ApplicationContainer echoapps = echoClient.Install(nodes.Get(0));
```

```
 echoapps.Start(Seconds(1.0));
 echoapps.Stop(Seconds(31.0));
if(trace){
 p2p.EnablePcapAll("p2p");
}
 AnimationInterface anim("que6.xml");
 anim.SetConstantPosition(nodes.Get(0), 10.0, 20.0);
 anim.SetConstantPosition(nodes.Get(1), 60.0, 20.0);
Simulator::Stop(Seconds(31.0));
Simulator::Run();
Simulator::Destroy();
return 0;
}
```

**Link for XML File :**

https://drive.google.com/drive/folders/1fqw0GkNkZQPWuagGgoi1B6_0IhGvH4-m?usp=drive_link

**Topology Diagram :**



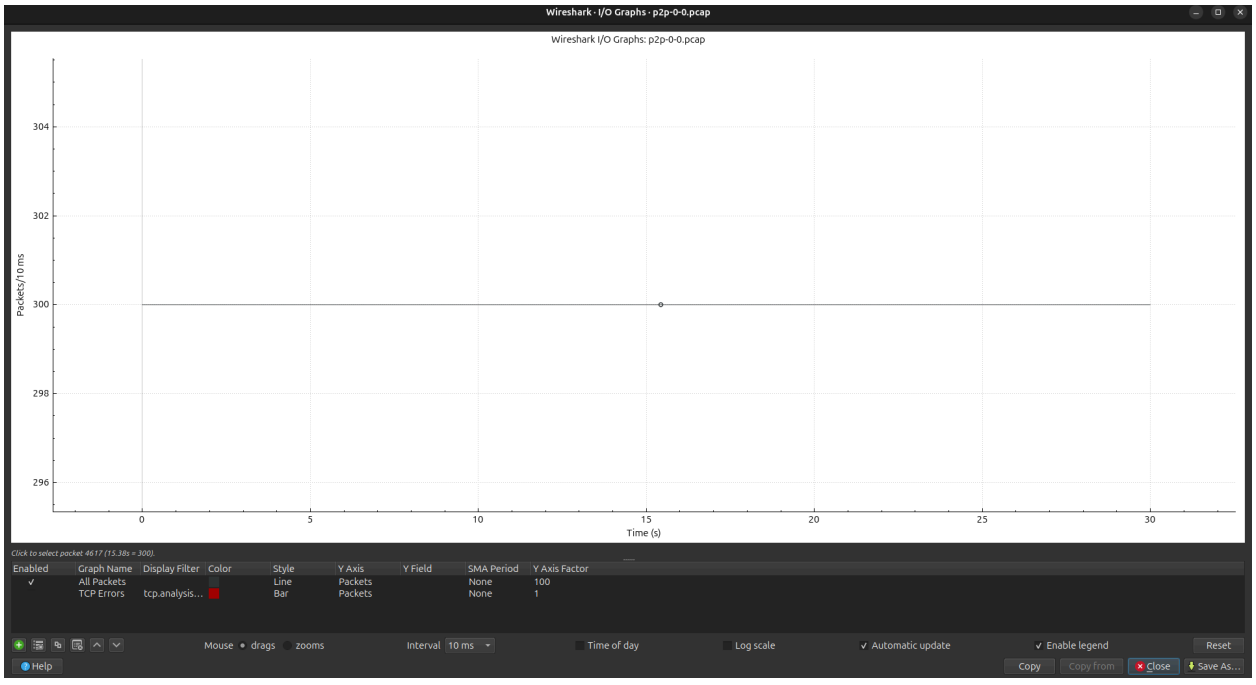

**Results :**

1. The nodes were positioned precisely at the coordinates defined in the code.

**Que 7.**

**Explanation :**
The analysis for this question extended the que 5 scenario by visualizing the network traffic flow using Wireshark I/O graph. The same PCAP files generated from the simulation ("p2p-0-0.pcap" and "p2p-1-0.pcap") were used to plot the I/O characteristics of the network interfaces.
The graph was configured to display data in bits/sec to illustrate bandwidth utilization, and the resulting visualization over the 30 second simulation successfully verified the continuous data transmission between Node 0 and Node 1.

**COMMAND USED :**
./ns3 build
./ns3 run “scratch/que7 --bandwidth=5Mbps --latency=2ms --maxpackets=4294967295 --interval=10ms --packetsize=1024”
**Link for pcap files :**
https://drive.google.com/drive/folders/1wEKCmAxdo8AEGFJjVTe5NTOrONGL1dw2?usp=drive_link
**Node 0:**

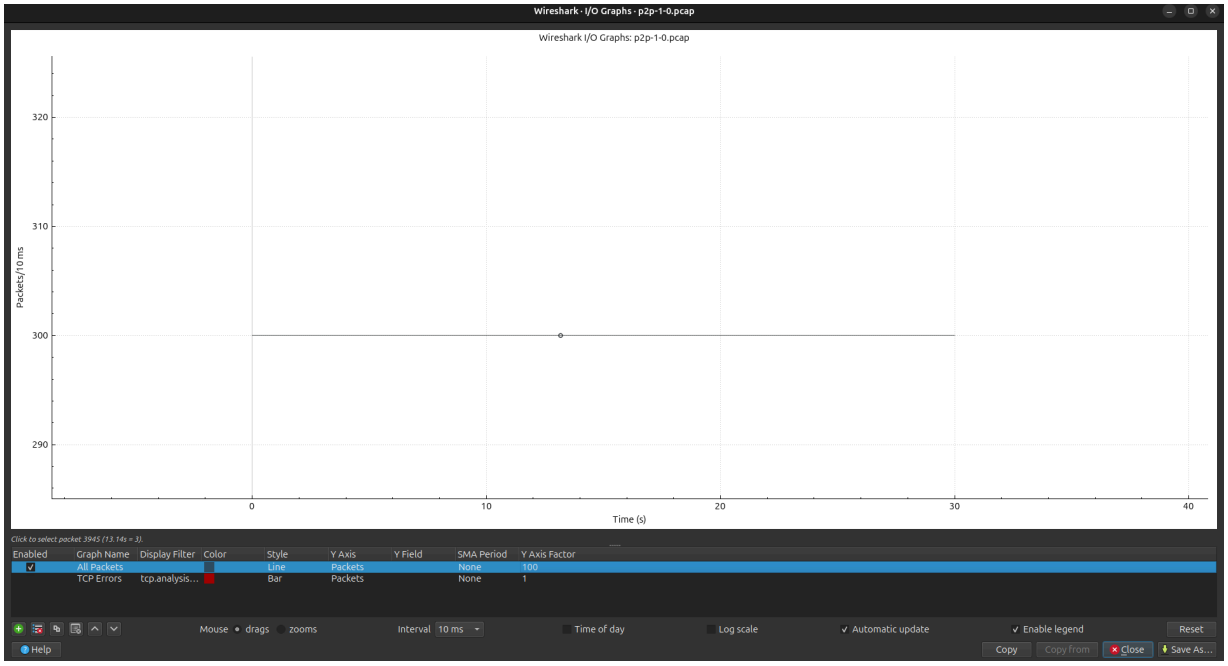


**Statistics**

| Measurement | Captured | Displayed | Marked |
|---|---|---|---|
| Packets | 9000 | 9000 (100.0%) | — |
| Time span, s | 29.999 | 29.999 | — |
| Average pps | 300.0 | 300.0 | — |
| Average packet size, B | 1054 | 1054 | — |
| Bytes | 9486000 | 9486000 (100.0%) | 0 |
| Average bytes/s | 316 k | 316 k | — |
| Average bits/s | 2,529 k | 2,529 k | — |

Capture file comments

## Node 1 :




**Statistics**

| Measurement | Captured | Displayed | Marked |
|---|---|---|---|
| Packets | 9000 | 9000 (100.0%) | — |
| Time span, s | 29.992 | 29.992 | — |
| Average pps | 300.1 | 300.1 | — |
| Average packet size, B | 1054 | 1054 | — |
| Bytes | 9486000 | 9486000 (100.0%) | 0 |
| Average bytes/s | 316 k | 316 k | — |
| Average bits/s | 2,530 k | 2,530 k | — |

Capture file comments

## Result :

1. The throughput remained constant at 300 packets/sec even under load.
2. The I/O graph shows no drops, confirming the network handled the traffic without congestion